

Marking instructions BAN404

Instructions

Your solution should be submitted as two files.

1. A pdf-file where both your R-code and the answers to the tasks are given.
2. Either a well documented R-script (.R) or an R markdown file (.Rmd) which can reproduce the results in the pdf-file.

The datasets used here are well analyzed before. Make sure to refer to other's work if you use it to solve the tasks.

A general recommendation: Be pragmatic if you run into difficulties. You need to find models that can produce predictions. Don't let the perfect prediction stand in the way for a good one!

The maximum total score on the exam is 100 and the number in brackets after each subtask shows the maximum score for that subtask.

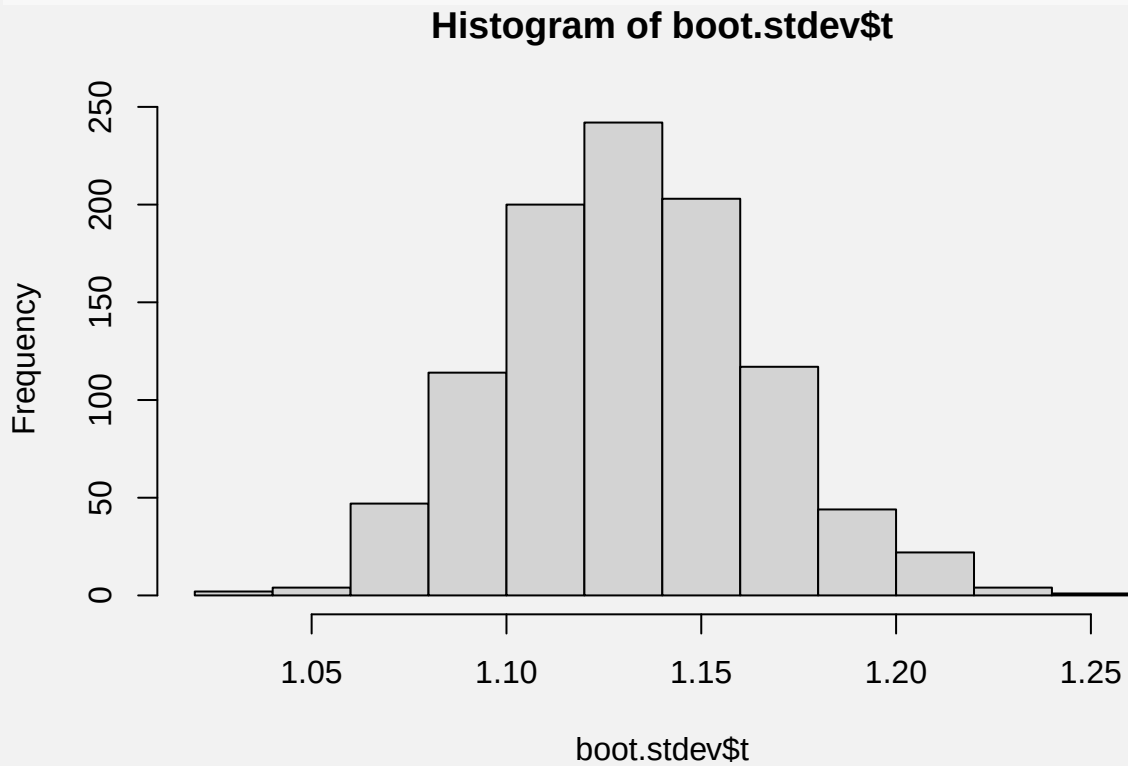
Task 1

In this task you will look at the **Smarket** data in the ISLR package.

- a) Use the bootstrap to create a histogram of the sampling distribution for the volatility of the S&P stock index, i.e. the standard deviation of the returns. Observations of the returns are in the variable **Today**. Start by setting the seed to 1, **set.seed(1)**. Use 1000 bootstrap replicates. (10)

Suggested solution:

```
set.seed(1)
library(ISLR)
library(boot)
ret=Smarket$Today
stdev=function(x,index) sd(x[index])
boot.stdev=boot(ret,stdev,R=1000)
hist(boot.stdev$t)
```



The task can also be solved by writing a loop and do the resampling by using the `sample`-function.

- b) Compute a 95% confidence interval based on the bootstrap, assuming that the volatility is normally distributed. (4)

Suggested solution:

```
boot.stdev

##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = ret, statistic = stdev, R = 1000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1* 1.136334 -0.004020027  0.03257308

# First way
orig=1.136334
se=0.03257308
c(orig-1.96*se,orig+1.96*se)

## [1] 1.072491 1.200177

# Another way
boot.ci(boot.stdev,type="norm")

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 1000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot.stdev, type = "norm")
##
## Intervals :
## Level      Normal
## 95%      ( 1.077,  1.204 )
## Calculations and Intervals on Original Scale
The boot.ci function adjusts for bias when type="norm". type="basic" produces the same result
as under "#First way" in the code above. All these answers give full score if done correctly.
```

- c) Compute a 95% confidence interval based on the bootstrap, not making the normality assumption but use percentiles from the bootstrapped volatilities. (5)

Suggested solution:

```
# First way
quantile(boot.stdev$t, probs=c(0.025, 0.975))

##      2.5%      97.5%
## 1.069278 1.200791

# Another way
boot.ci(boot.stdev, type="perc")

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 1000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot.stdev, type = "perc")
##
## Intervals :
## Level      Percentile
## 95%      ( 1.069,  1.201 )
## Calculations and Intervals on Original Scale
```

- d) Squared returns, Today^2 , can be used as a proxy for volatility. Squared returns are often autocorrelated, indicating that volatility can be predicted. Run a regression of the square of **Today** on the square of **Lag1** and interpret the estimate of the regression parameter for the square of **Lag1**. (5)

Suggested solution:

```
model1=lm(I(Today^2)~I(Lag1^2),data=Smarket)
summary(model1)

##
## Call:
## lm(formula = I(Today^2) ~ I(Lag1^2), data = Smarket)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.1869 -1.0864 -0.8310  0.0953 30.5847
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.07722    0.08362   12.882 < 2e-16 ***
## I(Lag1^2)    0.16511    0.02792    5.914 4.31e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.668 on 1248 degrees of freedom
## Multiple R-squared:  0.02726,    Adjusted R-squared:  0.02648
## F-statistic: 34.97 on 1 and 1248 DF,  p-value: 4.306e-09
If the volatility, measured as squared returns, at time  $t$  increases with one unit, we expect an volatility increase of 0.16 units at time  $t + 1$ . According to the regression output the coefficient is significantly different from zero.
```

- e) Compute the bootstrapped standard error of the regression coefficient in front of the square of **Lag1** in the model in d) and compare it with the standard error from the regression output. (Hint: An estimated regression coefficient from a simple linear regression with intercept made by `reg=lm(y~x)`

can be obtained by `reg$coefficients[2]`). (10)

Suggested solution:

```
volcoef=function(xdata,index) lm(I(Today^2)~I(Lag1^2),
                                data=xdata[index,])$coefficients[2]
boot.volcoef=boot(Smarket,volcoef,R=1000)
boot.volcoef

##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Smarket, statistic = volcoef, R = 1000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1*  0.1651066  0.003874388  0.04882499
```

The standard error is almost twice the size of the standard error in the regression output.

Task 2

In this task you will look at the data set `dataCar`, vehicle insurance policies, in the `insuranceData` package. You will find a description of the data set using the help function `help(dataCar)` and the link to the source for the data mentioned there. `exposure` is a measure of the risk the insurer takes on for a insurance taker. You should predict the claim amount, `claimcst0`, given that an insurance policy has at least one claim.

- a) Start by first selecting the observations where there is an occurrence of claim, `clm!=0`. Also, remove the variable `X_OBSTAT_`. Work with this new data set for the rest of Task 2. (3)

Suggested solution:

```
library(insuranceData)
data(dataCar)
xdata=dataCar[dataCar$clm!=0,]
xdata=xdata[, -11]
```

- b) Use descriptive statistics, the description of the data in the help-function and common sense to remove variables that you think will not be helpful. Motivate your choices thoroughly. Also, make sure that all variables are on the right format. (7)

Suggested solution:

```
sapply(xdata,class)
```

```
## veh_value  exposure      clm numclaims claimcst0  veh_body  veh_age  gender
## "numeric" "numeric" "integer" "integer" "numeric" "factor" "integer" "factor"
##      area  agecat
## "factor" "integer"
```

```
xdata=xdata[,-3]
xdata$numclaims=xdata$numclaims
xdata$agecat=as.factor(xdata$agecat)
xdata$veh_age=as.factor(xdata$veh_age)
```

Remove `clm` since it is contained `numclaims`. `exposure` could potentially be predicted for a new insurance taker but might not be observable. Could be removed or kept. `veh_age`, `agecat` recoded to be factors since they represent age categories such as “youngest”.

- c) Use a cross validation approach to evaluate predictions for the following models: A linear regression with all predictors left in the dataset after 2b, all possible simple (one-predictor) linear regression models and finally, a model without any predictors (a linear regression of `claimcst0` on an intercept). You should use a CV method that estimates the `testMSE` as precisely as possible at the same time as being feasible to implement. Motivate your choice of CV-method. If random draws are, somehow, used in your analysis, first set the seed to 543, `set.seed(543)`. (14)

Suggested solution:

```
set.seed(123)
n=nrow(xdata)
p=ncol(xdata)-1 # Number of predictors
K=5
testMSES=matrix(0,K,p+2)
xvars=names(xdata)[-4]
colnames(testMSES)=c("full",xvars,"naive")
fold_size=floor(n/K)
ii=1
ind=sample(1:n,size=n) # Scramble the data to avoid
                        # unintentional sorting

ydata=xdata[ind,]
for(i in 1:K)
{
  training=ydata[-(ii:(ii+fold_size-1)),]
  test=ydata[(ii:(ii+fold_size-1)),]
  ii=ii+fold_size
  regfull=lm(claimcst0~.,data=training)
  predfull=predict(regfull,newdata=test)
  testMSES[i,1]=mean((test$claimcst0-predfull)^2)
  for(k in 1:8)
  {
    form1=formula(paste("claimcst0~",xvars[k]))
    regsimp=lm(form1,data=training)
    predsimp=predict(regsimp,newdata=test)
    testMSES[i,k+1]=mean((test$claimcst0-predsimp)^2)
  }
  naive=mean(training$claimcst0)
  testMSES[i,k+2]=mean((test$claimcst0-naive)^2)
}
colMeans(testMSES)

##      full veh_value exposure numclaims veh_body  veh_age   gender    area
## 12277335 12610027 12389564 12538918 12633044 12605801 12579658 12605616
## agecat    naive
## 12580305 12606509
```

A first attempt with leave-one-out CV revealed that this is a time consuming problem. The 4624 observations means that $4624 \times 9 = 41616$ linear regressions had to be estimated. The second attempt with 5-fold CV turned out to be less problematic since only 180 models had to be estimated. The full regression gave the smallest testMSE, 12277335.

- d) Use a generalized additive model (GAM) with smoothing splines for the numerical variables. Include categorical variables in the model as well. You can use the `s`-function in the `gam`-library, with argument `df=4`, without motivation. Plot the result and comment on the relationship between the predictors and the dependent variable. (7)

Suggested solution:

```
library(gam)
```

```
## Loading required package: splines
```

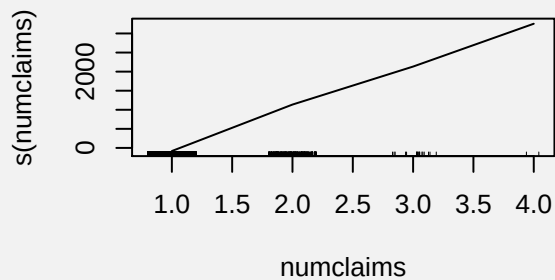
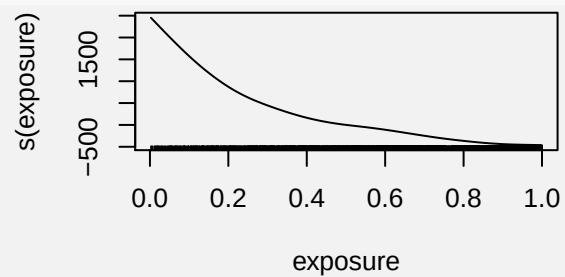
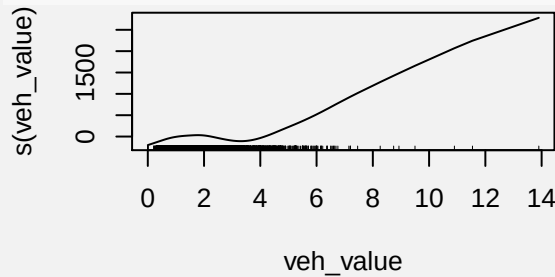
```
## Loading required package: foreach
```

```
## Loaded gam 1.20
```

```
gam1=gam(claimcst0~s(veh_value)+s(exposure)+s(numclaims)+  
          veh_body+veh_age+gender+area+agecat,data=xdata)
```

```
par(mfrow=c(2,2))
```

```
plot(gam1,terms=c("s(veh_value)","s(exposure)","s(numclaims)"))
```



The marginal effect of `numclaims` looks linear indicating that the second and third claim is not systematically different in size compared to the first. The marginal effect of `veh_value` is non-linear for small values but linear for larger values. Note that there are less than one percent of claims where the vehicle value is larger than 6 so the non-linearity is more relevant than the picture might imply. `exposure` is non-linear and negatively related to claim size.

- e) Use your conclusions in 2d to specify an appropriate GAM-model. Compute the `testMSE` for this GAM-model. (4)

Suggested solution: Use the code from 2c and add the new model.

```
set.seed(123)
n=nrow(xdata)
p=ncol(xdata)-1 # Number of predictors
testMSES=matrix(0,K,p+3)
xvars=names(xdata)[-4]
colnames(testMSES)=c("full",xvars,"naive","fullgam")
K=5
fold_size=floor(n/K)
ii=1
ind=sample(1:n,size=n) # Scramble the data to avoid
                        # unintentional sorting

ydata=xdata[ind,]
for(i in 1:K)
{
  training=ydata[-(ii:(ii+fold_size-1)),]
  test=ydata[(ii:(ii+fold_size-1)),]
  ii=ii+fold_size
  regfull=lm(claimcst0~.,data=training)
  predfull=predict(regfull,newdata=test)
  testMSES[i,1]=mean((test$claimcst0-predfull)^2)
  for(k in 1:8)
  {
    form1=formula(paste("claimcst0~",xvars[k]))
    regsimp=lm(form1,data=training)
    predsimp=predict(regsimp,newdata=test)
    testMSES[i,k+1]=mean((test$claimcst0-predsimp)^2)
  }
  naive=mean(training$claimcst0)
  testMSES[i,k+2]=mean((test$claimcst0-naive)^2)
  gam1=gam(claimcst0~s(veh_value)+s(exposure)+numclaims+
            veh_body+veh_age+gender+area+agecat,data=training)
  predgam=predict(gam1,newdata=test)
  testMSES[i,k+3]=mean((test$claimcst0-predgam)^2)
}
colMeans(testMSES)

##      full veh_value exposure numclaims veh_body  veh_age   gender    area
## 12277335 12610027 12389564 12538918 12633044 12605801 12579658 12605616
##      agecat    naive    fullgam
## 12580305 12606509 12243092
```

Since there was no evidence on a non-linear relation between `numclaims` and the dependent variable, linearity is imposed for this variable. The full `gam` model has a marginally smaller `testMSE` than the full linear regression.

Task 3

In this task we will continue to work with the `dataCar` data but now focus on predicting the variable `clm`. Note that the starting point is the original `dataCar`, not the version you created in Task 2.

- a) Start by removing the variables `X_OBSTAT_`, `numclaims` and `claimcst0`. Compute and interpret appropriate cross-tables of `clm` and the categorical variables. Also compute and interpret well-chosen

descriptive statistics (such as central tendencies and measures of variation) for the numerical variables for each of the two categories of `clm`. Are some of the categories of the categorical variables more interesting than others? Based on this, do you see any promising predictors for `clm`? Mention a drawback with such pairwise comparisons. (7)

Suggested solution:

The proportion of claims for all policy takers are 6.8 % so when comparing for sub-categories in the data, this is a number to remember. The proportion of claims is not very different for the different vehicle type. If something, BUS (19 %) and MCARA (11 %) stands out as more probable to have claims. Vehicle age seem to have little to say about the existence of a claim. There are no big differences between male and female insurance takers or between different areas either. The youngest drivers though have a relative claim frequency of 8.6 % compared to for all observed policy takers. Concerning the numerical variables, policies with claims has, on average, a larger vehicle value and a higher exposure than the ones without. We could of course proceed to test for significant differences between the groups but that is not required to obtain full score. `veh_body`, `agecat`, `veh_value` and `exposure` looks promising. By using pairwise comparisons we will not detect interactions effects of predictors.

- b) Based on your hypotheses from 3a, use logistic regression based on all or a subset of the predictors to predict `clm`. Interpret the estimated coefficients. (7)

Suggested solution:

```
logit1=glm(clm~veh_body+agecat+veh_value+exposure,data=xdata,family=binomial())
summary(logit1)
```

```
##
## Call:
## glm(formula = clm ~ veh_body + agecat + veh_value + exposure,
##      family = binomial(), data = xdata)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -0.9614  -0.4243  -0.3300  -0.2656   2.8707
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -2.38497    0.38070  -6.265 3.73e-10 ***
## veh_bodyCONVT -2.02915    0.71494  -2.838 0.004537 **
## veh_bodyCOUPE -0.68210    0.39809  -1.713 0.086637 .
## veh_bodyHBACK -1.03022    0.37784  -2.727 0.006400 **
## veh_bodyHDTOP -0.92717    0.38798  -2.390 0.016861 *
## veh_bodyMCARA -0.50951    0.47536  -1.072 0.283791
## veh_bodyMIBUS -1.14154    0.40890  -2.792 0.005243 **
## veh_bodyPANVN -0.99943    0.39988  -2.499 0.012444 *
## veh_bodyRDSTR -1.08263    0.83319  -1.299 0.193813
## veh_bodySEDAN -1.02896    0.37769  -2.724 0.006443 **
## veh_bodySTNWG -1.04359    0.37828  -2.759 0.005801 **
## veh_bodyTRUCK -1.13229    0.38867  -2.913 0.003577 **
## veh_bodyUTE   -1.26787    0.38216  -3.318 0.000908 ***
## agecat2       -0.20684    0.05896  -3.508 0.000451 ***
## agecat3       -0.26068    0.05748  -4.535 5.75e-06 ***
## agecat4       -0.29431    0.05734  -5.132 2.86e-07 ***
## agecat5       -0.51339    0.06375  -8.053 8.10e-16 ***
## agecat6       -0.51723    0.07287  -7.098 1.27e-12 ***
## veh_value      0.06002    0.01366   4.393 1.12e-05 ***
## exposure      1.86056    0.05483  33.932 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 33767  on 67855  degrees of freedom
## Residual deviance: 32423  on 67836  degrees of freedom
## AIC: 32463
##
## Number of Fisher Scoring iterations: 6
```

For the `agecat` variables, all other categories have a lower propensity to have claims than the youngest group. The coefficient -0.20684 , e.g., means that the odds of a claim for the second youngest group, for given values of all other predictors, is $\exp(-0.20684) \approx 81\%$ of the odds for the youngest group. Similar interpretation for the `veh_body` where BUS is the reference group. The coefficient for `veh_value`, 0.06002 means that, everything else equal, if the vehicle value increases with 10000 dollar, the odds for a claim would, on average, be $\exp(0.06002) \approx 106\%$ of the odds before the increase. Similar interpretation for the `exposure` variable.

- c) Modify the model if you believe it to be sensible. Use the validation set approach with a 50/50 training/test split to evaluate the predictions from the logistic regression. Choose a threshold for when an estimated probability should lead to a prediction of a claim. You do not have to use a formal method but imagine a decision problem, using the prediction, that relies on the assumption that it costs more to predict a claim as a “non-claim” than vice versa. Compute a confusion matrix and argue for your choice of threshold in the context of a trade-off between false positive and false negative predictions. Set the seed to 8712, `set.seed(8712)`. (10)

Suggested solution:

```
set.seed(1234)
n=nrow(xdata)
ind=sample(1:n,size=floor(n/2))
training=xdata[ind,]
test=xdata[-ind,]
logit1=glm(clm~veh_body+agecat+veh_value+exposure,data=training,family=binomial())
prob1=predict(logit1,newdata=test,type="response")
pred1=prob1>0.07
prop.table(table(test$clm,pred1),margin=1)

##      pred1
##      FALSE      TRUE
## 0 0.6293903 0.3706097
## 1 0.4071398 0.5928602
```

Setting a threshold should preferably be made by experimenting with predictions in the training data. Starting with the threshold 0.5 implies that we always predict a new policy as a “no claim”. Lowering the threshold eventually means that we will make more incorrect predictions. However this might be worthwhile if the cost of a false negative is higher than the cost of a false positive.

- d) Use boosted trees to predict `clm` and evaluate the predictions with the same approach as in 3c. (10)

Suggested solution:

```
library(gbm)

## Loaded gbm 2.1.8

gbm1=gbm(clm~veh_body+agecat+veh_value+exposure,
          data=training,distribution="bernoulli")
probgbm=predict(gbm1,newdata=test,type="response")

## Using 100 trees...

pred2=probgbm>0.07
prop.table(table(test$clm,pred2),margin=1)

##      pred2
##      FALSE      TRUE
## 0 0.5638955 0.4361045
## 1 0.3285168 0.6714832
```